

Mock Interview Series

Field Nation — Full-Stack Software Engineer

Komol Chandra Devnath · Prepared 2026-09-29

1. [Series Overview](#)
2. [Mock 1 — CV Deep Dive & Behavioral](#)
3. [Mock 2 — System Design](#)
4. [Mock 3 — Full-Stack](#)
5. [Mock 4 — DB + API](#)
6. [Mock 5 — Rapid-Fire & Fluency](#)

5-Mock Interview Series — Field Nation SE

The target is a 1–2 hour technical interview in English with a US-style team. The mocks run from easy to hard.

#	File	Focus	Status
1	<code>mock-01-cv-behavioral.md</code>	CV deep dive + behavioral STAR	● prep file ready · fill in the <code>[FILL]</code> gaps, then run it live
2	<code>mock-02-system-design.md</code>	System design + distributed systems (was FN-20, the full loop)	● template exists
3	<code>mock-03-fullstack.md</code>	Node / Laravel / React / FastAPI deep dive	▢ stub (prompt only)
4	<code>mock-04-db-api.md</code>	DB optimization + API engineering (was FN-19, technical)	● template exists
5	<code>mock-05-rapid-fire.md</code>	Rapid-fire + spoken fluency drill	▢ stub (prompt only)

Master Setup Prompt (paste first, in every mock)

Act as a Senior Technical Interviewer evaluating me for a Full-Stack Software Engineer role. - **CV:** `/home/kiri2ka/code/sokrio/_komol/InterviewLens/Komol_CV_FieldNation_Software_Engineer.docx`
- **Format:** 1–2 hour deep-dive interview - **Goal:** Assess technical accuracy, system architecture knowledge, communication clarity, and spoken English fluency.

1. Ask **one question at a time**.
2. After I answer, give feedback on **technical depth, clarity/conciseness**, and **spoken English/grammar**, then rate my answer (1–10).
3. Show an optimized STAR or technical answer before moving on to the next question.

Rules for every mock

- Answer **out loud first** and record yourself, then type what you said.
- After each mock: fill in its Fix List, and copy every item scored 6 or lower into `_memory/weak-areas.md`.
- Don't start the next mock until the previous mock's top 3 fixes are practiced.

Mock 1 — CV Deep Dive & Behavioral (60–75 min)

Goal: Defend every line of your CV in clear, confident spoken English, and answer behavioral questions with STAR. **Rule for this mock:** Every number you say must be one you can explain. If a line says [FILL], write the real answer before the mock. **Never invent a number.**

How to run it

1. Before the mock, fill in every [FILL] in this file (about 30–45 min). Say each model answer out loud **once**, but don't memorize it word for word. Memorized answers sound robotic.
2. Paste the **Live Mock Prompt** (bottom of this file) into Claude. Claude asks one question at a time.
3. Answer out loud first (record yourself on your phone), then type what you said.
4. After the mock, fill in the **Fix List** and update _memory/weak-areas.md.

Time box

Time	Section	What they are really checking
0–5 min	Warm-up small talk	Are you relaxed and friendly?
5–10 min	"Tell me about yourself"	Can you summarize 5 years in 90 seconds?
10–50 min	CV deep dive (Part C)	Did <i>you</i> really do this? How deep does it go?
50–65 min	Behavioral STAR (Part D)	Will you be a good teammate on a US/offshore team?
65–70 min	Your questions	Are you genuinely interested?

Part A — Warm-up (5 min)

They say	You say
"Hi Komol, how are you doing today?"	"I'm doing great, thanks for asking! How about you?"
"Where are you joining from?"	"I'm in Dhaka, Bangladesh. It's [evening] here, so it's a nice quiet time for a call."
"Can you hear me okay?"	"Yes, loud and clear. Please let me know if my audio breaks up at any point."
"Did you find the link easily?"	"Yes, no problem at all. Thanks for setting it up."

Tip: Smile, speak **slower than feels natural**, and always ask a question back ("How about you?").

Part B — "Tell me about yourself" (90 seconds)

Structure: Present → Past highlights → Why this role

"Sure. I'm a full-stack software engineer with over five years of backend experience in PHP, Laravel and Node.js, and about three years of React and TypeScript.

Right now I work at **Sokrio**, a multi-tenant SaaS for distribution and order management. It's used by more than 100 enterprise clients. A few things I'm proud of: I **led the migration from a Laravel monolith to event-driven microservices** over RabbitMQ with zero downtime, I made our **heaviest reports up to 85% faster**, and I'm the **primary on-call engineer** for six production apps, where we use SLOs to catch problems early.

I also work a lot with clients and distributed teams in English, through Slack, Jira and PR reviews.

I'm excited about Field Nation because the domain is very close to what I've built: I built **field-force modules at Sokrio**, with task tracking, attendance and GPS-spoofing detection. And your move toward Node.js microservices is exactly the kind of work I've done."

Follow-ups to expect: - "Why are you leaving Sokrio?" → *"I've learned a lot there, and I'm proud of what we built. Now I want a bigger engineering team, a larger scale, and a product where my field-force experience matters directly."* **Never complain about your current company.** - "What are you learning right now?" → *"Go, system design and SRE practices. I'm also getting deeper into NestJS, which I'd use for Node microservices."*

Part C — CV Deep Dive (40 min)

Every block follows the same shape: **Opening question** → **Model answer (STAR, 90–120 s)** → **Drill questions** → **Trade-off line** → **Numbers to know** → **Key phrases**

Priority: ● = they will almost certainly ask · ● = likely · ● = maybe, keep it short

C1 ● **Monolith** → **Event-driven Microservices (S1)**

Opening: "Tell me about the biggest technical project you've led."

Model answer:

S: "At Sokrio we had a Laravel 8 and Vue.js 2 monolith. [FILL: the real pain, e.g. slow deploys, one heavy report slowing down the whole app, teams blocking each other.] **T:** I was asked to lead the migration to a more scalable architecture, **without any downtime**, because 100+ clients use it every day. **A:** We split it into a Laravel core REST API, two separate React apps (one for tenants, one for admin), and three services (Report, Notification and SOP) that talk over **RabbitMQ**, with

MongoDB and Redis. The key decision was to **run both systems in parallel** and move traffic step by step, so we could roll back at any point. **R:** We cut over with zero downtime, and server load dropped by about 20%."

Drill questions: 1. "How did you decide the service boundaries?" → *"We split along parts that change or scale independently. Reports were heavy and bursty, and notifications were async by nature, so both were natural candidates."* 2. "How did you keep data in sync while both systems were running?" → [FILL: dual-write? events? a shared DB during the transition?] 3. "How did you measure the 20%?" → [FILL: CPU? requests per second? which dashboard?] 4. "What happens if RabbitMQ goes down, or a message is processed twice?" → *"Consumers are idempotent, messages that fail go to a dead-letter queue with retries, and important events are durable."* 5. "What would you do differently?" → *"Add distributed tracing earlier. Debugging across services was harder than inside the monolith."*

Trade-off line: *"Microservices gave us independent scaling and deploys, but the cost is operational complexity: more things to monitor, network failures, and eventual consistency."*

Numbers to know: team size [FILL] · duration [FILL] · ~20% server-load reduction (measured how: [FILL])

Key phrases: "we ran both systems in parallel" · "a zero-downtime cutover" · "service boundaries" · "eventual consistency"

C2 Reports up to 85% Faster (S2)

Opening: *"Tell me about a performance problem you solved."*

Model answer:

S: "Our heaviest reports ran over millions of records. [FILL: which report, e.g. a sales or order summary.] It took about [FILL: X] seconds, and sometimes timed out for large clients. **T:** I owned making it fast and reliable. **A:** First I **measured** instead of guessing: I ran EXPLAIN on the slow queries. I added **composite indexes**, removed **N+1 queries**, and moved the heavy aggregation reads to **MongoDB aggregation pipelines**. Large reports became **queued jobs** with retries, a dead-letter queue and priorities, and results were **cached in Redis**. **R:** Load time dropped by up to 85%, from [FILL: X]s to [FILL: Y]s, and report generation stays at 99.9%+ availability."

Drill questions: 1. "What did EXPLAIN show you?" → [FILL: e.g. a full table scan, type: ALL, "Using filesort"] 2. "Which composite index, and why that column order?" → *"The equality columns first, then the range or sort column, because of the leftmost-prefix rule."* 3. "How do you invalidate the cache?" → *"Keys are tenant-scoped and have a TTL. When the underlying data changes, we clear the related keys."* [FILL: event-based or TTL-only?] 4. "Why MongoDB and not just better MySQL?" → *"The aggregation reads were heavy, read-only and denormalized, which fits documents well. MySQL stayed the source of truth for transactions."*

Trade-off line: *"Caching makes reports fast, but the trade-off is freshness. Users might see data that's a few minutes old, so we agreed the TTL with the product team."*

Numbers to know: up to 85% faster ([FILL: before → after]) · 99.9%+ availability · "millions of records" ([FILL: roughly how many?])

Key phrases: "I measured before optimizing" · "a full table scan" · "leftmost prefix" · "dead-letter queue"

C3 ● On-call, SLIs/SLOs, 65% Faster Resolution (S4)

Opening: "Tell me about a production incident you handled."

Model answer:

S: "I'm the primary 24/7 on-call engineer for six production apps. [FILL: one real incident: what broke, when, and who was affected.] **T:** I had to restore service fast and make sure it didn't happen again. **A:** [FILL: how you detected it (which alert or dashboard), how you found the root cause (logs, Telescope, APM), the fix, and how you communicated with the team and clients during the incident.] Afterwards I [FILL: added an alert, a test or a runbook]. **R:** More broadly, when we moved from raw CPU alerts to **SLO-based alerts**, for example 99.9% report availability, API latency and error rate, our issue resolution time dropped by 65%."

Drill questions: 1. "What's the difference between an SLI, an SLO and an SLA?" → "An SLI is the measurement, like the error rate. An SLO is our internal target, like 99.9%. An SLA is the promise to the customer, with consequences if we miss it." 2. "How was the 65% measured?" → [FILL: mean time to resolve, before vs. after? over what period?] 3. "How do you avoid alert fatigue?" → "Alert on user-facing symptoms (SLO burn), not on every CPU spike."

Trade-off line: "A 99.99% target sounds nice, but every extra nine costs a lot of engineering. 99.9% was the right balance for reports."

Key phrases: "root cause" · "a blameless postmortem" · "error budget" · "we alert on symptoms, not causes"

C4 ● Vue → React/Next/RTK/TypeScript Rewrite: Leadership (S6)

Opening: "Tell me about a time you led a team or set technical standards."

Model answer:

S: "Our frontend was on Vue.js, and [FILL: why rewrite? e.g. hard to hire for, inconsistent code, performance problems]. **T:** I led the rewrite to React, Next.js, Redux Toolkit and TypeScript, and we had to finish it in **two months**. **A:** First, I **cut the scope** to the key user journeys. I worked with UX designers and product to rebuild them from Figma. Then I set the **standards**: component structure, folder layout and code-review rules, and documented them so the team could follow them without me. [FILL: team size, how you split the work.] **R:** We shipped in two months, the team adopted the standards, and user engagement went up 30%."

Drill questions: 1. "How did you fit a rewrite into two months?" → *"Scope. We rebuilt the most-used journeys first, not every screen."* [FILL] 2. "How did you measure the 30% engagement increase?" → [FILL: which metric? daily active users? session time? which tool?] 3. "What if someone on the team didn't agree with your standards?" → *"I'd listen first. If their idea was better, we'd change the standard. The goal is team consistency, not my preference."* 4. "Why Redux Toolkit and not just Context?" → *"Complex, shared server state across many pages. RTK gave us predictable updates and good devtools."*

Trade-off line: *"A rewrite is risky, because you stop shipping features for a while. We reduced that risk by keeping the scope small and releasing journey by journey."*

Key phrases: "I cut the scope" · "the team adopted the standards" · "we worked closely with UX and product"

C5 Multi-tenancy + Payments + POS (S3, S5, SoftArch)

Opening A: *"How does multi-tenancy work in your system?"*

"Each client company gets its **own tenant database**, and a central **landlord database** holds platform data like organizations and billing. When a request comes in, middleware resolves the tenant and switches the database connection. On top of that, **every cache key and every queued job carries the tenant scope**, and we have **territory-scoped RBAC**, so users only see their own region's data. Across 100+ tenants, we've had zero cross-tenant data leaks."

- Drill: "How does a queued job know which tenant it belongs to?" → *"The tenant ID is part of the job payload. The job switches the connection before it runs and switches back afterwards."*
- Drill: "What's the downside of database-per-tenant?" → *"Migrations must run on 100+ databases, and cross-tenant reporting needs a loop or a separate analytics store."*
- Drill: "Why not a shared DB with a `tenant_id` column?" → *"That's cheaper, but one missed WHERE clause can leak data. For enterprise clients, strong isolation was worth the cost."*

Opening B: *"How did you make payments reliable?"*

"I designed the payment gateway integration for billing and subscriptions: initiation, webhooks, retries, auto-reconciliation and receipts. Two things make it safe. **Every webhook is verified with an HMAC signature**, so nobody can fake a 'payment succeeded' call. And **every request carries an idempotency key**, so if the gateway retries or sends a duplicate callback, we never charge twice. A **reconciliation job** catches anything that drifts. We've had zero failed transactions in production."

- Drill: "What if the webhook never arrives?" → *"Reconciliation polls the gateway for pending payments and fixes the status."*
- Drill: "Which gateway?" → [FILL]

Opening C: *"Tell me about GirlyShopper."*

"At SoftArch I built GirlyShopper, a multi-store e-commerce platform with **POS integration**, barcode inventory and real-time stock tracking, using Laravel, Vue and MySQL. It cut manual stock reconciliation by about 60%. I also integrated card, bKash, Nagad and cash-on-delivery payments, with webhook-driven order status updates."

- Drill: "How did you keep stock correct between online orders and the POS?" → [FILL: row locks? atomic decrement?] *Hint: "an atomic UPDATE with `stock >= qty` in the WHERE clause, so two sales can't oversell."*

Key phrases: "strong isolation" · "HMAC signature verification" · "idempotency key" · "reconciliation"

C6 REST/SOAP APIs + Client Integrations (S7)

Opening: "Tell me about working with a difficult client integration."

S/T: "I designed the REST APIs behind our mobile field-force app, plus SOAP endpoints and queue-based sync for legacy enterprise clients. [FILL: one specific client problem, e.g. their data kept failing to sync.] **A:** [FILL: how you debugged it (logs, request IDs, reproducing their payload), and how you communicated: a short call, then a written summary.] **R:** [FILL: it was fixed, and what you changed so it doesn't happen again.]"

Drill: "How do you explain a technical problem to a non-technical client?" → "Impact first, then cause, then what we're doing and when it'll be fixed. No jargon."

C7 Field-force Modules + GPS-Spoofing Detection (S9), your "Why Field Nation" story

Opening: "What feature are you most proud of?"

"I built the field-force side of Sokrio: **Task Manager** (plan, approval, achievement tracking), **Van Sales**, **Vehicle Feedback** and **Van Attendance**, all syncing in real time with the core platform. The interesting part was **GPS-spoofing detection**, because field staff could fake their location to mark attendance. We detected it by [FILL: mock-location flag from the device? impossible speed or distance jumps between points?]. These modules contributed to the company's 27% revenue growth. And this is why Field Nation excites me: technician check-in, location proof and work-order approval are the same kind of problem."

Careful: Say "contributed to 27% revenue growth", not "caused". Be ready for: "What was *your* part of that 27%?"

C8 ● Node.js Services

Opening: "You list Node.js. What did you build with it?"

"Alongside the Laravel core, I built Node.js/Express services, background workers and developer tooling in JavaScript and TypeScript. [FILL: one concrete example, e.g. a worker that consumes RabbitMQ events.]"

Drill: "PHP vs. Node: when would you choose each?" → "Node shines for I/O-heavy, concurrent work like real-time events and many open connections, because of its event loop. Laravel is very productive for CRUD-heavy business apps."

C9 ● Docker + CI/CD (~40% faster deployments)

Opening: "Tell me about your deployment process."

"I containerized our services with Docker and built CI/CD pipelines that run automated tests before every deploy. Deployments got about 40% faster, and we stopped having 'works on my machine' environment mismatches."

⚠ **Flag:** The 40% has no source story yet. [FILL: how was it measured? minutes before vs. after? which CI tool, e.g. GitHub Actions or GitLab CI?] If you can't explain it, say "noticeably faster" instead of the number.

C10 ● Short answers (30–60 s each)

Question	Short answer
"How do you use AI in your work?" (S8)	"I use Claude Code every day, with a custom setup of agents for database, backend, frontend and tests, and project rules they must follow. I review every diff myself , and tests are required, so AI makes me faster without lowering quality."
"Do you practice TDD?"	"For new service and API logic, yes. I write the test first with PHPUnit or pytest. Tests are a required phase of every feature, so regressions get caught in CI, not in production."
"Tell me about Am Coders."	"I built and shipped a multi-tenant SaaS sold on CodeCanyon, with RBAC, a REST API, a Vue frontend and a full subscription lifecycle. I handled buyer support and usually fixed issues within 24 hours. I started as an intern and was promoted to Software Engineer within a month."
"Your CV mentions research."	"I have two papers under review: emotion detection from customer reviews using LLMs, and a statistical analysis of suicide case patterns in Bangladesh. It taught me to work carefully with data and to explain results clearly."
"You list NestJS / Kubernetes / Go. How strong are you?"	Be honest: "NestJS I'm actively learning. It's close to what I know from Express and Laravel's structure. Kubernetes I'm familiar with, but I haven't run it in production. Go I'm learning now."

Part D — Behavioral STAR (15 min)

STAR timing: S+T = 20 seconds · **A = 60 seconds (the most time, and say "I", not "we")** · R = 15 seconds with a number · Lesson = 10 seconds

#	Question	Story	Status
D1	"Tell me about a time you showed leadership."	S6 (C4 above)	✅ ready
D2	"Tell me about a time you worked under pressure."	S4 (C3 above)	🟡 needs a real incident
D3	"Tell me about a difficult client or stakeholder."	S7 (C6 above)	🟡 needs a real case
D4	"Tell me about a time you disagreed with a teammate or manager."	S10, new	📄 fill in below
D5	"Tell me about a mistake you made."	S11, new	📄 fill in below

D4 — Disagreement (S10) scaffold

Guiding questions to help you remember: Did you ever disagree about a technology choice (MongoDB? RabbitMQ? React?), an estimate, the scope of a feature, or a code-review comment?

S: "[FILL: when, and who (use a role like 'a senior colleague' or 'our product manager', not a name)]" **T:** "We disagreed about [FILL]." **A:** "I first **listened to understand their reasons**. Then I [FILL: showed data, built a small proof of concept, or wrote down both options with pros and cons]. We agreed to [FILL]." **R:** "[FILL: the outcome.] What I learned is that disagreements get resolved faster with data than with opinions."

Good signal: You stay respectful, use data, and are willing to be wrong. **Red flag:** "I was right and they were wrong."

D5 — Mistake / failure (S11) scaffold

Guiding questions: a bad deploy? a migration that broke something? a wrong estimate? a bug that reached production?

S: "[FILL: what you did wrong. Own it clearly: 'I made a mistake when...']" **A:** "I [FILL: fixed it fast, told the team openly]. Then I [FILL: added a test, a checklist or an alert] so it can't happen again." **R:** "[FILL.] Since then I always [FILL: the habit you built]."

Good signal: real ownership plus a system-level fix. **Red flag:** blaming others, or a fake mistake ("I work too hard").

Part E — Your questions (5 min)

Pick 2 from 04-behavioral/03-questions-to-ask.md. The best ones for this mock: - "How far along is the move from PHP to Node.js microservices, and is it done piece by piece?" (connects to S1) - "What does success look like for this role in the first 3 months?"

CV Risk Check — every number on your CV

Interviewers in the US often ask "How did you measure that?" If you can't answer, use the safe phrasing.

CV claim	How was it measured?	Safe phrasing if unsure
Reports up to 85% faster	[FILL: before → after seconds, which report]	"up to 85% on our heaviest reports"
99.9%+ report-generation uptime	[FILL: SLO dashboard, over what period?]	"we held our 99.9% SLO"
Issue resolution 65% faster	[FILL: MTTR before vs. after?]	"resolution time dropped significantly, about 65%"
27% company revenue growth	[FILL: your modules' role]	" contributed to 27% revenue growth"
User engagement +30%	[FILL: which metric and tool?]	"engagement rose around 30% after launch"
Server load ~20% lower	[FILL: CPU? requests per second?]	"about 20% lower server load"
Deployments ~40% faster	⚠️ [FILL: no source story yet]	"noticeably faster deployments"
Stock reconciliation ~60% less	[FILL]	"much less manual reconciliation, about 60%"
100+ tenants, zero leakage	Tenant count from the landlord DB	✅ safe
Zero failed transactions	[FILL: over what period?]	"no failed transactions in production"

Useful English for this mock

Buying time: "That's a great question, let me think for a second." · "Let me give you some context first." **Clarifying:** "Just to make sure I understand, are you asking about X or Y?" · "Sorry, could you repeat the last part?" **Owning your part:** "I was responsible for..." · "My role specifically was..." · "The team did X, and I personally did Y." **Honest gaps:** "I haven't used that in production, but here's how I'd approach it..." **Finishing an answer:** "...so that's how we solved it. Happy to go deeper into any part."

Common grammar slips to watch for: - "I have done it **in** 2023" → "I **did** it in 2023" (a finished time → simple past) - "We are using Redis **since** 2023" → "We **have been using** Redis since 2023" - "He told **to me**" → "He **told me**" - "Discuss **about** the design" → "Discuss the design" - Plurals: "many **clients**", "two **services**", "millions of **records**"

Scoring rubric (1–10 per answer)

Area	1-3	4-6	7-8	9-10
Technical depth	vague, buzzwords	correct but shallow	specific, with trade-offs	specific, trade-offs, and handles drill questions
STAR structure	no structure	partial, missing R	complete STAR	complete, with a number and a lesson
Clarity / length	lost or rambling	too long or too short	60-120 s, clear	crisp and easy to follow
English	hard to follow	understandable, many slips	natural, few slips	fluent and confident
Ownership ("I")	only "we"	mixed	clear "I"	clear "I" and credits the team

Live Mock Prompt (paste into Claude)

Act as a Senior Technical Interviewer from a US company, evaluating me for a Full-Stack Software Engineer role at Field Nation. My CV is at `/home/kiri2ka/code/sokrio/_komol/InterviewLens/Komol_CV_FieldNation_Software_Engineer.docx` and my prep file is `study/03-roles/fieldnation-software-engineer/05-mocks/mock-01-cv-behavioral.md`.

Let's begin **Mock Interview 1: CV Deep Dive & Behavioral**. Start with a short warm-up, then "tell me about yourself", then CV deep-dive questions (C1-C9) with realistic follow-ups, then 3 behavioral questions (D1-D5). **Ask one question at a time**. After each answer: 1. Rate me 1-10 using the rubric in the prep file 2. Give feedback on technical depth, clarity/conciseness, and spoken English/grammar 3. Show a corrected, more natural version of what I said 4. Show the ideal STAR or technical answer Then ask the next question. At the end, give an overall score and my top 3 fixes.

Fix List (fill in after the mock)

Date	Question	Score	What went wrong	Fix action	Closed?

Tracker ID: FN-20 · Generated: 2026-09-28 · Source: `_memory/story-bank.md`, `04-behavioral/01-pitch-why-fieldnation.md`, `04-behavioral/02-hr-answers.md`, `03-system-design/01-work-order-dispatch.md`, `03-system-design/02-php-to-node-migration.md`, `backend-system-design-study-plan.md` Phase 12 Case 2

🔴 2026-09-29: Renamed from `mock-02-full-loop.md` → this is now **Mock 2 of 5 (System Design)** in the new series (D8). The behavioral part moved to `mock-01-cv-behavioral.md`; focus on §২-৩ here.

Mock 2 prompt: "Let's begin **Mock Interview 2: System Design & Architecture**. Focus on distributed systems, microservices, caching (Redis), message queues (RabbitMQ), database scaling (MySQL/MongoDB aggregations), and cloud deployment (AWS/Docker). Challenge my architectural choices and ask follow-ups on performance bottlenecks and high availability. Ask one question at a time."

Mock 2 — System Design (was Full Loop, ৯০ মিনিট)

এটাও একটা **টেমপ্লেট** (Day 13-এ চালানোর জন্য), ফলাফল আগে থেকে বানানো না। রেকর্ড করো, স্কোর দাও, নিচের Fix list ভরাট করো।

টাইম বক্স

সময়	সেকশন
২০ মিনিট	Behavioral
৪০ মিনিট	Coding (২টা DSA + ১টা backend design-in-code)
৩০ মিনিট	System Design — Technician Matching/Routing

১. Behavioral (২০ মিনিট — প্রতিটা ≤২ মিনিট)

- "Tell me about your biggest project." → S1 (monolith → event-driven microservices)
- "Tell me about a production incident you handled." → S4 (on-call + SLO, ৬৫% দ্রুত রেজোলিউশন)
- "Why Field Nation?" → `04-behavioral/01-pitch-why-fieldnation.md`-এর "Why FN" উত্তর (কেন্দ্রে S9)
- (ঐচ্ছিক ৪র্থ, সময় থাকলে) "A time you disagreed with a teammate/manager" → `story-bank`-এ এখনো নেই, [Komo1 পূরণ করবে: S10 হিসেবে যোগ করো Day 13-এর আগে]

২. Coding (৪০ মিনিট)

DSA #১ (~১৫ মিনিট) — Trees প্যাটার্ন

একটা বাইনারি ট্রি দেওয়া আছে; level-order traversal-এ প্রতিটা লেভেলের গড় মান রিটার্ন করো (TypeScript-এ)।

DSA #২ (~১৫ মিনিট) — Graphs BFS প্যাটার্ন

একটা adjacency list হিসেবে technician-দের "একই region"-এ থাকা গ্রাফ দেওয়া আছে; BFS দিয়ে একটা নির্দিষ্ট technician থেকে ৩-হপের মধ্যে সবাইকে বের করো।

Backend design-in-code (~১০ মিনিট)

Work Order status-কে TypeScript discriminated union দিয়ে মডেল করো (draft | published | assigned | checked_in | checked_out | approved | paid | cancelled), আর একটা transition(current, next) ফাংশন লেখো যেটা অবৈধ ট্রানজিশনে একটা কাস্টম error (409-সমতুল্য) থ্রো করে। [Sokrio link](#): সরাসরি অভিজ্ঞতা নেই এই ঠিক ফর্মে — এভাবে বলো: "Sokrio-তে order/task status-এর মতো state machine সামলেছি Eloquent lifecycle hooks দিয়ে; TS discriminated union একই আইডিয়া, কম্পাইল-টাইমে চেক করে দেয়।"

৩. System Design — Technician Matching / Routing (৩০ মিনিট)

প্রসঙ্গ: "একটা নতুন Work Order পোস্ট হলে, তার আশেপাশের সেরা ২০ জন যোগ্য technician খুঁজে বের করে নোটিফাই করো।"

Requirements clarify করো: skill/certification filter, distance radius, availability, rating, একই buyer-এর সাথে আগের কাজ, blocked list।

Deep dives (অন্তত ২টা বেছে বলো): - Geo filtering: geohash vs Redis GEOADD / GEOSearch vs MySQL spatial index vs OpenSearch geo query — কোনটা কখন - Ranking score: precompute vs per-request compute; fairness (একই technician-কে বারবার পিং না করা) - Async flow: WorkOrderPublished event → matching worker → ProvidersNotified (এখানে S1-এর RabbitMQ অভিজ্ঞতা টানো)

[Sokrio link](#): S1 (event-driven pipeline), S9 (technician/field-force ডোমেইন পরিচিতি) — কিন্তু বড়-স্কেল geo-matching নতুন টপিক, সততার সাথে স্বীকার করো।

স্কোরিং রুব্রিক (০-৩, [weak-areas.md](#) -এর স্কেলের সাথে মিলিয়ে)

মাপকাঠি	০	১	২	৩
Clarity	এলোমেলো	মূল পয়েন্ট আছে কিন্তু গোছানো না	পরিষ্কার, মাঝে মাঝে থামে	শুরু থেকে শেষ পর্যন্ত গোছানো
Correctness	ভুল	আংশিক সঠিক	সঠিক, ছোট গ্যাপ	সম্পূর্ণ সঠিক
Trade-off reasoning	বলেইনি	১টা trade-off বলেছে	২টা+ trade-off, তুলনা করেছে	trade-off + কখন ভিন্ন সিদ্ধান্ত নিত সেটাও বলেছে

মাপকাঠি	০	১	২	৩
Communication (English)	থেমে থেমে	বোঝা যায় কিন্তু slow	flowing, occasional pause	interview-ready pace

Fix List (মক চালানোর পর ভরাট করো)

তারিখ	কোন প্রস্নে/সেকশনে আটকেছি	স্কোর	ফিক্স অ্যাকশন	বন্ধ?

Tracker ID: FN-23 · Stub created: 2026-09-29 · Series: Mock 3 of 5 · Full content to be generated before running it

Mock 3 — Full-Stack Technical Deep Dive

Reuse: 02-topics/01-javascript-es6.md , 02-topics/02-typescript.md , 02-topics/03-nodejs-nestjs.md , 02-topics/06-php-laravel-delta.md , 02-topics/07-react-redux-ts.md

Prompt

Let's begin **Mock Interview 3: Full-Stack Technical Deep Dive**. Focus on core engineering concepts: - **Backend:** event loop, async processing, API design, middleware, ORM performance. - **Frontend:** React/Next.js state management, rendering strategies (SSR/SSG), performance optimization. Ask technical questions with follow-ups that test edge cases and optimization strategies. Ask one question at a time.

Fix List

Date	Question	Score	What went wrong	Fix action	Closed?

Tracker ID: FN-19 · Generated: 2026-09-28 · Source: 02-topics/01-javascript-es6.md, 02-topics/02-typescript.md, 02-topics/04-mysql-advanced.md, 02-topics/05-rest-api-webhooks.md

🔥 2026-09-29: Renamed from mock-01-technical.md → this is now **Mock 4 of 5 (DB + API)** in the new series (D8). See 00-series-overview.md.

Mock 4 prompt: "Let's begin **Mock Interview 4: Database Optimization & API Engineering**. Ask 5 scenario-based questions about schema design, slow query optimization, indexing strategies, RESTful API design, background processing pipelines, and data synchronization between systems. Ask one question at a time."

Mock 4 — DB + API (৬০ মিনিট)

এটা একটা **টেমপ্লেট**, রেকর্ডেড ট্রান্সক্রিপ্ট না। Day 12-এ আসলে চালাও (টাইমার নিয়ে, জোরে বলে), তারপর নিচের "Fix list" ভরাট করো।

কীভাবে চালাবে

- একা হলে জোরে বলো + রেকর্ড করো; কেউ থাকলে তাকে interviewer বানাও।
- প্রতিটা সেকশনের টাইমার আলাদাভাবে চালাও, ওভার হলেও থামবে না — কোথায় আটকেছ সেটাই আসল ডেটা।
- Pass মানে:** কোড কম্পাইল-লেভেলে সঠিক (সিনট্যাক্স এরর ছাড়া লজিক), কেন এই অ্যাপ্রোচ সেটা বলতে পারা, আর একটা ফলো-আপ "যদি X হয়?" প্রশ্নে আটকে না যাওয়া।

টাইম বক্স

সময়	সেকশন	সোর্স ডক
৫ মিনিট	ইন্ট্রো + ৩০-সেকেন্ড pitch	04-behavioral/01-pitch-why-fieldnation.md
২০ মিনিট	JS/TS live coding (২টা প্রবলেম)	02-topics/01-javascript-es6.md, 02-topics/02-typescript.md
১৫ মিনিট	MySQL — index + EXPLAIN	02-topics/04-mysql-advanced.md
১৫ মিনিট	REST/Webhook ডিজাইন প্রশ্ন	02-topics/05-rest-api-webhooks.md
৫ মিনিট	Wrap-up + প্রশ্ন জিজ্ঞাসা	04-behavioral/03-questions-to-ask.md

১. JS/TS Live Coding (২০ মিনিট)

Q1 (~১০ মিনিট) — `debounce` টাইপড ভার্সন

TypeScript-এ একটা জেনেরিক `debounce<T extends (...args: any[]) => void>(fn: T, wait: number): T` লেখো। তারপর ব্যাখ্যা করো: এটা `setTimeout` আর `event loop`-এর `macrotask queue` কীভাবে ব্যবহার করে, আর `throttle`-এর সাথে পার্থক্য কী।

Q2 (~১০ মিনিট) — জেনেরিক `groupBy`

`function groupBy<T, K extends string | number>(items: T[], keyFn: (item: T) => K): Record<K, T[]>` লেখো। একটা ফলো-আপ: `Work Order` অ্যারেকে `status`-এর ওপর `group` করে দেখাও। এরপর জিজ্ঞাসা করবে: `unknown` বনাম `any` কোথায় ব্যবহার করতে, `generics`-এর সুবিধা কী।

২. MySQL — Index + EXPLAIN (১৫ মিনিট)

Q1 — কম্পোজিট ইনডেক্স ডিজাইন

এই কোয়েরিটার জন্য ইনডেক্স ডিজাইন করো (জোরে বলে, `leftmost-prefix` যুক্তিসহ): `SELECT * FROM work_orders WHERE buyer_id=? AND status=? ORDER BY scheduled_at DESC LIMIT 20`

Q2 — Accept Race Condition

দুইজন technician একই মিলিসেকেন্ডে একটা `Work Order`-এ "Accept" ক্লিক করলে কী হয়? তিনটা সমাধান বলো (`conditional UPDATE` + `affected_rows` চেক, `SELECT ... FOR UPDATE`, `version column`) — সবচেয়ে সহজটা কেন সবচেয়ে ভালো এখানে সেটাও বলো। [🔗 Sokrio link: S2 \(MySQL/EXPLAIN গভীরতার প্রমাণ, direct race-condition experience না — সংভাবে বলো\)](#)

৩. REST/Webhook ডিজাইন (১৫ মিনিট)

Design করো: `workorder.status_changed` webhook একটা `buyer`-এর ইন্টিগ্রেশনে পাঠানো — `payload shape`, `retry policy`, `HMAC signature verification`, `replay protection`। জিজ্ঞাসা করবে: `webhook` দুইবার এলে কী হয়, `gateway` ডাউন থাকলে কী হয়। [🔗 Sokrio link: S5 \(payment webhook HMAC + idempotency — একদম সরাসরি প্রাসঙ্গিক\)](#)

Fix List (মক চালানোর পর ভরাট করো)

তারিখ	কোন প্রশ্নে আটকেছি	কেন আটকেছি	ফিক্স অ্যাকশন	বন্ধ?

Tracker ID: FN-24 · Stub created: 2026-09-29 · Series: Mock 5 of 5 · Full content to be generated before running it

Mock 5 — Rapid-Fire & Spoken Fluency Drill

Reuse: 02-topics/05-rest-api-webhooks.md, 02-topics/10-docker-k8s-aws-linux.md, 99-final-cram.md

Prompt

Let's begin **Mock Interview 5: Rapid-Fire & Fluency Drill**. Ask 8 shorter, high-yield questions covering language internals, HTTP protocols, security best practices (JWT, CORS, OAuth), and CI/CD pipelines. Focus heavily on how fluently, concisely, and confidently I explain complex concepts. Ask one question at a time.

Fix List

Date	Question	Score	What went wrong	Fix action	Closed?